©2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

From Engineering Diagrams to Graphs: Digitizing P&IDs with Transformers

1st Stürmer, Jan Marius
German Aerospace Center (DLR)
Institute for the Protection of Terrestrial Infrastructures
Sankt Augustin, Germany
jan.stuermer@dlr.de

2nd Graumann, Marius
German Aerospace Center (DLR)
Institute for the Protection of Terrestrial Infrastructures
Sankt Augustin, Germany
marius.graumann@dlr.de

3rd Koch, Tobias
German Aerospace Center (DLR)
Institute for the Protection of Terrestrial Infrastructures
Sankt Augustin, Germany
tobias.koch@dlr.de

arXiv:2411.13929v3 [cs.CV] 19 Dec 2025

***Abstract*—Digitizing engineering diagrams like Piping and Instrumentation Diagrams (P&IDs) plays a vital role in maintainability and operational efficiency of process and hydraulic systems. Previous methods typically decompose the task into separate steps such as symbol detection and line detection, which can limit their ability to capture the structure in these diagrams. In this work, a transformer-based approach leveraging the Relationformer that addresses this limitation by jointly extracting symbols and their interconnections from P&IDs is introduced. To evaluate our approach and compare it to a modular digitization approach, we present the first publicly accessible benchmark dataset for P&ID digitization, annotated with graph-level ground truth. Experimental results on real-world diagrams show that our method significantly outperforms the modular baseline, achieving over 25% improvement in edge detection accuracy. This research contributes a reproducible evaluation framework and demonstrates the effectiveness of transformer models for structural understanding of complex engineering diagrams. The dataset is available under https://zenodo.org/records/14803338.**

***Index Terms*—engineering diagrams, digitization, graph reconstruction, computer vision.**

I. Introduction

The design process of complex technical systems starts with a conceptual engineering drawing that describes the properties of hydraulic components and instrumentation as well as their interconnections. These properties and relationships can be represented by an attributed, directed graph or relational structured data formats. Machine-readable information about technical systems can be used to support planning and operation of these systems through simulation using state-of-the-art frameworks. Nowadays, such plans are created digitally using CAD software and stored as machine-readable data. However, there is a large old stock of such plans in non-digital form. Furthermore, the exchange of such plans often takes place via rendered images or PDF files. This is linked to a rising demand for digitization of such documents. One type of diagrams are Piping and Instrumentation Diagrams (P&ID). A P&ID is a detailed diagram used in the chemical process industry, which describes the equipment installed in a plant, along with instrumentation, controls, piping etc. and is used during planning, operation and maintenance [1].

This paper presents several contributions to the field of P&ID digitization. Firstly, we describe methods for digitizing P&IDs based on a recent transformer network along with the modular digitization approach based on previous work. We describe the metrics used for evaluating the performance of our proposed method, providing a comprehensive framework for assessing its effectiveness. We then compare our proposed methods using synthetically created and real-world engineering diagrams. To facilitate further research and development, we publish the small test dataset *PID2Graph*. Notably, this dataset is, to the best of our knowledge, the first publicly available P&ID dataset that contains real-world data and annotations for the full graph, including symbols and their connections.

II. Related Work

Engineering diagram digitization primarily involves two key tasks: extracting components and their interconnections. The successful digitization of P&IDs enables the automated creation of accurate simulation models, as demonstrated by previous research. For instance, [2] generated models from text-based descriptions, while [3] developed a pipeline for generating simulation models directly from digitized engineering diagrams. P&ID digitization can be approached in two ways: as a series of separate sub-problems solved by one module per sub-problem or as an image-to-graph problem.

A. Modular Engineering Diagram Digitization

The most commonly employed method to digitize P&IDs is utilizing separate deep learning models or algorithms for symbol detection, text detection, and line detection. The connection detection usually is not evaluated. This approach was first proposed by [4] and has since been refined in subsequent work [5, 6]. Here, Convolutional Neural Networks (CNN) are employed for symbol and text detections. Afterwards, probabilistic hough transform or other traditional computer vision algorithms are applied to detect lines. To combine these detections, a graph describing the components and their interconnections is created. A recent review by [7] provides an in-depth analysis of existing literature on deep learning-based engineering diagram digitization. In contrast to employing CNNs

©2025 IEEE. Appears in *2025 IEEE 12th International Conference on Data Science and Advanced Analytics (DSAA)*,
DOI: 10.1109/DSAA65442.2025.11248012

for symbol detection (e.g. , [8, 9]), alternative approaches have been explored, including the use of segmentation techniques [10] or Graph Neural Networks (GNNs) [11]. Additionally, other studies have focused on related but distinct tasks, such as line and flow direction detection [12] or line classification [13].

The principles of engineering diagram digitization can be extended to other domains, even if the application domain or diagram type differs from P&IDs. A recent study by [14] has demonstrated the digitization of process flow diagrams (PFDs), leveraging a dataset comprising approximately 1,000 images sourced from textbooks. The connections and graph were extracted by skeletonizing the image. However, the connection extraction was not evaluated due to missing labels for this task. Other approaches deal with electrical engineering diagrams [15, 16], handwritten circuit diagram digitization [17], mechanical engineering sketches [18] or construction diagrams [19].

B. Image to Graph

Another possibility to tackle engineering diagram digitization is framing it as an image-to-graph problem. [20] introduce a road graph extraction algorithm that is able to extract a graph describing a road network from satellite images using neural networks. A similar problem is described by [21], where a *Generative Graph Transformer* is described to extract graphs from images of road networks. Although the problem of identifying connections in diagrams is similar to extracting connections from engineering diagrams, the challenges of identifying specific symbols and extracting different types of relationships remain. The Scene Graph Generation (SGG) task involves solving these challenges, where objects from an image are extracted and the relationships between them are determined [22]. One transformer network dealing with SGG is SGTR+ [23]. A recent framework combining SGG and road network extraction is the *Relationformer* proposed by [24]. The Relationformer, an enhancement of deformable DETR (DEtection TRansformer) [25], combines advanced object detection with relation prediction and is shown to outperform existing methods. Since its release, the Relationformer has successfully been adapted for further work like crop field extraction [26]. Metrics that have been used to evaluate graph extraction evaluation are manifold. For the problem of road network extraction from images, the metrics Streetmovers-Distance [21], TOPO [27, 21] or APLS Metric [28] were introduced. Another commonly used relation metric is the scene graph generation metric mR@K. Objects are matched by IOU (or similar metrics) and the (object, relation, object)-tuples are ranked by confidence. The K most confident results are then used to calculate the mean recall. For scene graph generation, this approach with using only the top-K results is used because the annotations are highly incomplete due to the high amount of possible relations. [29]

TABLE I: Classes used for training and evaluation.

Symbols			Lines
General	Pump/Compressor	Arrow	Solid
Tank/Vessel	Instrumentation	Inlet/Outlet	Non-Solid
Valve			

C. Datasets

Developing effective models for P&ID digitization requires P&IDs that are accurately labeled with symbols and connections. Unfortunately, existing research has mostly been evaluated on private datasets, which have not been published due to concerns around copyright and intellectual property rights. Some approaches use synthetic data [8, 30] or augment their data applying Generative Adversarial Networks (GANs) [31]. The only published dataset for P&ID digitization is Dataset-P&ID by [30] consisting of 500 synthetic P&IDs with 32 different symbol classes. The data includes annotations for symbols, lines, and text, which are valuable resources. However, the graph structure annotations are not provided, and the lines between symbols are drawn using a simplified grid layout. Additionally, one lines style can change between dashed and solid forms. These limitations may impact the effectiveness of certain models or methods trained or evaluated with this dataset.

III. Methods

Due to its supposed ability to be adaptable, we apply the Relationformer model in the context of engineering diagram digitization, and compare it to a method that breaks down the digitization task into separate sub-tasks of detecting symbols, text, lines, and then extracting graphs, referred to as *Modular Digitization Approach* in this paper. Both the proposed Relationformer and the Modular Digitization Approach share a common goal of identifying and classifying symbols and lines within P&IDs. Their outputs are unified in the form of a graph representation, where each node contains a bounding box and symbol class, while edges have an edge class. The models will be trained and evaluated with the classes listed in Table I, i.e., seven symbol classes and two line classes. The workflow for the preprocessing step and both methods is visually depicted in Fig. 1 and are discussed in detail in the subsequent sections.

A. Preprocessing

Due to the high resolution of P&IDs alongside with big size differences between symbols, the input resolution for the models would have to be high in order to still accurately depict symbols and lines. We use patching to split the full diagram resized to 4500x7000 into multiple patches with an overlap of at least $50\,\%$. During both model processing steps, each patch is independently processed and then combined with its neighbors to form a complete graph representation.

B. Relationformer

The Relationformer is described as in the original paper [24] and implemented in a modified form to adapt to P&IDs.

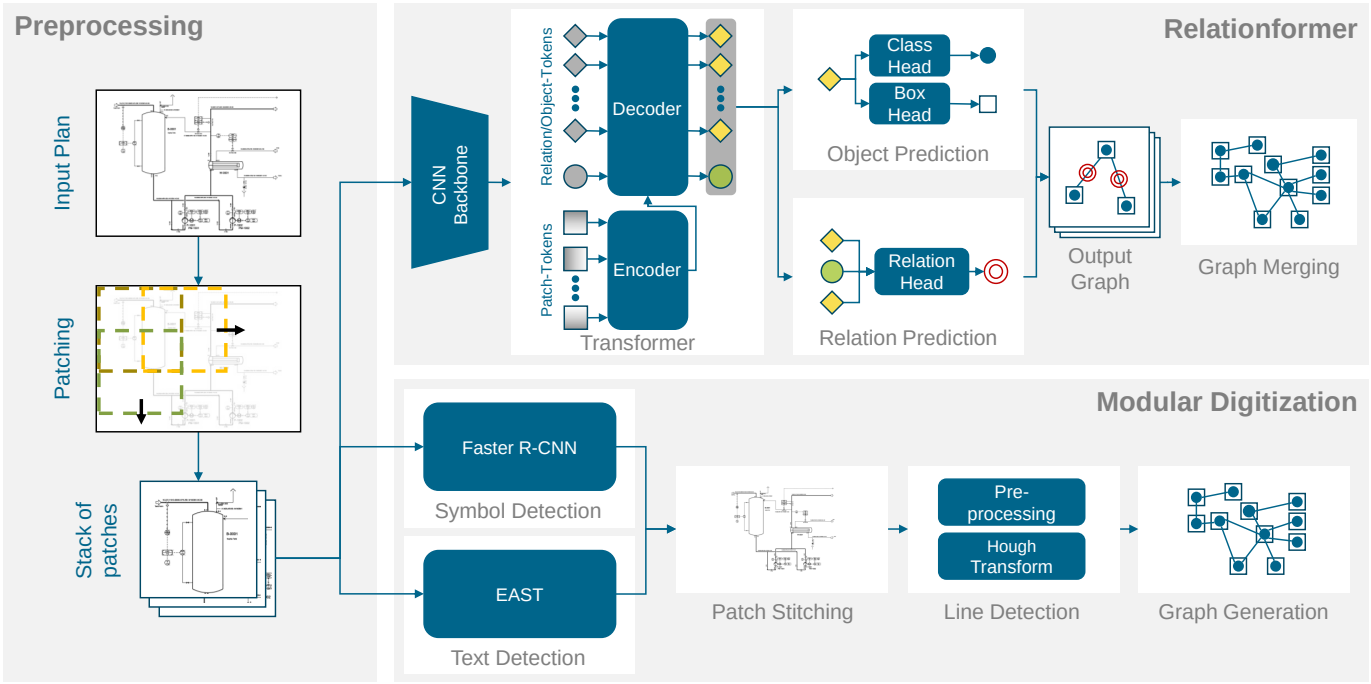
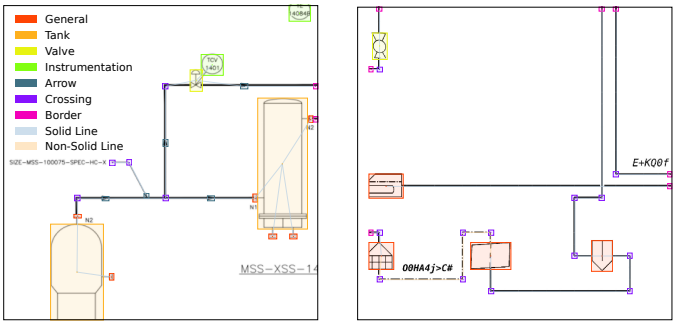


Fig. 1: Overview of the Relationformer [24] and the Modular Digitization to digitize engineering diagrams. The preprocessing step patches and adjusts the data, which is then fed into each method to produce a graph representation as output.

The Relationformer is based on deformable DETR [25] and consists of a CNN backbone, a transformer encoder-decoder architecture and heads for object detection and relation prediction (see Fig. 1). The main difference to deformable DETR is the decoder architecture and the relation prediction head. The decoder uses $N + 1$ tokens as the first input, where N is the number of object-tokens and a single relation-token. The second input are the contextualized image features from the encoder. The objection detection head consists of two components. Firstly, Fully Connected Networks (FCNs) are employed to predict the location of each object within the image, in form of a bounding box that define the spatial extent of each object. Secondly, a single layer classification module is used, assigning a class label to each detected object. Thus, the output of the object detection head consists of the class and bounding box of the object. The relation prediction head has a pairwise object-token and a shared relation-token, where a multi layer-perceptron (MLP) predicts the relation $\tilde{e}^{ij}_{\text{rln}} = \text{MLP}_{\text{rln}}(o^i, r, o^j{}_{i \neq j})$ for every object-token pair o^i and o^j with the relation-token r.

To adapt the Relationformer model for training on the patched data, several modifications were made to the input graph. In addition to the pre-existing symbol classes, new node categories were introduced to capture key features of the diagram's structure: line *ankles*, *crossings* and *borders*. Nodes get appended to the graph accordingly with a bounding box around the center of the node that has a uniform size, analogously to the procedure described in the original Relationformer paper regarding road networks. A border bounding box is created when a line gets cut during patching at the position where



(a) (b)

Fig. 2: Two example patches obtained after dividing the full diagram of OPEN100 (a) and the Synthetic Test Data (b), with border nodes (pink) and bounding boxes (several colors) marking where lines exit each patch. These patches serve as input to the Relationformer for training, testing and evaluation.

the line intersects the border of the patch. Examples for this patched images and graphs can be seen in Fig. 2.

Afterwards, the predicted graphs for single patches are merged into a graph representing the complete plan. The merge process consists of the following steps:

1) Lowering the confidence score for each bounding box in a patch with the function $\hat{c} = c - \alpha \cdot e^{-3|d\frac{2}{S}|}$, where $\hat{c}$ is the modified confidence score, c is the original confidence score, S is the size of the patch, d is the smallest distance between the bounding box and the patch border and α is the maximal amount of the

weighting $\alpha = 0.4$. Therefore, symbols with bounding boxes closer to the patch border are more likely to be cut off, as there is a possibility the same symbols is included to a bigger extent in another patch;
2) Filtering predictions with a low confidence score in order to ignore them during the merging process;
3) Collecting and resizing all bounding boxes for the complete plan;
4) Non-maximum suppression (NMS) with a high IOU threshold to merge duplicates;
5) Weight-boxed fusion (WBF) with a lower IOU threshold to combine information from bounding boxes;
6) Cleaning up the graph by removing self-loops and non-connected nodes.

C. Modular Digitization Approach

An alternate, more commonly used approach involves separately detecting symbols, text and lines and then merging them together into a graph, following earlier work of [3], as can be seen in Fig. 1.

a) Symbol Detection: An improved Faster R-CNN [32] is used for symbol detection, with patching and merging done as described in Section III-B. Because no graph structure is considered by this object detector, the classes for crossings, ankles and border nodes do not exist.

b) Text Detection: Text is identified using CRAFT [33] and EasyOCR [34]. Notably, this text detection functionality serves as an auxiliary mechanism for filtering purposes only, rather than being evaluated. Filtering text prevents the line detection module from incorrectly classifying textual elements as graph connections.

c) Line Detection: Lines are detected using dilation, erosion, and Progressive Probabilistic Hough Transform. Dashed lines are reconstructed by clustering and merging small line segments.

d) Graph Generation: The final step generates a comprehensive graph by assigning line start and end points to symbols or other lines, creating crossing and ankle nodes as needed, and refining the graph by removing self-loops and unused nodes.

D. Datasets

To address the gap of missing complex and real-world datasets, we have created our own datasets with annotations for symbols and connections. The synthetic data is generated using symbol templates scrambled and cutout from P&ID standardization and legends. Our algorithm randomly places these templates on a canvas and connects them in a way that forms a connected graph. If lines intersect, a crossing node is created. Furthermore, this data includes various line types, such as solid and dashed lines, to enhance the realism of our digitization task and add data diversity. To further simplify the data and focus on the essential graph structure, additional information such as tables, legends and frames are removed. The patched training dataset is augmented with various transformations, comprising small angle rotations, 90° rotations, horizontal and vertical flips, minor adjustments to brightness and contrast, random scaling, and image blurring.

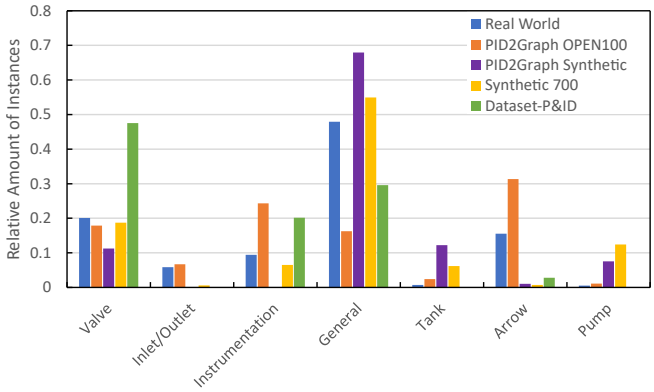


Fig. 3: Symbol class distribution: Frequency of symbol object classes among the datasets, showing relative abundance of each class type.

The training data consist of synthetic and real-world data. *Synthetic 700* is a synthetic dataset with 700 different symbol templates collected from a broad range of sources. Additionally, we have manually annotated 60 *real-world P&IDs* from various plants.

For the *PID2Graph Synthetic* dataset, synthetic plans composed exclusively of symbols from [35], which are based on the ISO 10628 standard [36], are included.

PID2Graph OPEN100 is used to further validate our method. It includes 12 manually annotated publicly available P&IDs from the OPEN100 reactor [37]. Notably, these plans do not contain dashed lines, allowing us to assess the robustness of our model in a scenario where this feature is absent.

To enable a comparison with previous work, we also evaluate our methods on *Dataset-P&ID* from [30]. This dataset includes annotations in the form of bounding boxes for 32 different symbols and start and end points of line segments, rather than in a graph format. We convert these annotations to align with the format used in our other datasets. Specifically, we map the 32 symbol classes to our classes as outlined in Table I, and we connect overlapping line segments to create edges. Since these edges may include both dashed and solid segments, we assign a unified edge label across all edges.

The relative distribution of symbol classes, excluding crossings and ankles, is depicted in Fig. 3. A notable pattern emerges across all datasets: the 'general' class overwhelmingly dominates the distribution in every dataset except OPEN100. This can be attributed to the fact that the general class comprises a broad range of symbols, resulting in its disproportionate representation.

The relative distribution of edge classes is shown in Fig. 4. The non-solid edge class makes up between 10% and 20% of all edges in the datasets except OPEN100.

E. Metrics

We evaluate the quality of graph construction using object detection metrics, where each detected bounding box

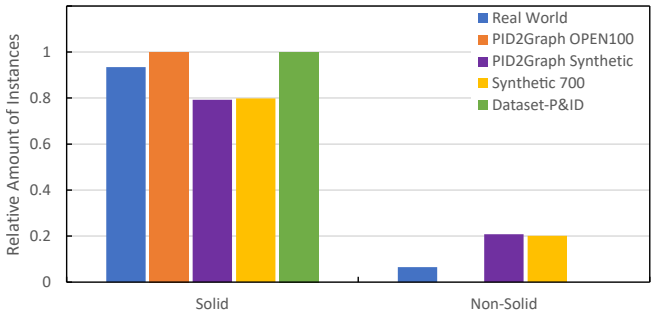


Fig. 4: Edge Class Distribution: Relative frequency of each edge class across the datasets.

corresponds to a node in the graph along with a metric for measuring edge detection performance.

1) Node Detection: To comprehensively measure the performance of our method in constructing graph nodes, we employ two metrics. Firstly, mean Average Precision (mAP) is used to evaluate the estimation of individual symbols, considering only classes from Table I that do not involve crossings and ankles, as these categories relate to graph connectivity rather than symbol representation.

The second metric used is average precision (AP) across all symbols, crossings and ankles. However, unlike the first metric, each symbol is assigned to the same class, because an error or a confusion in the assignment of the class to a node is not decisive for the entire graph reconstruction, as long as the node exists. This allows us to evaluate the consistency and accuracy of our method in constructing the graphs structure. Both metrics are calculated using an Intersection over Union (IOU) threshold of 0.5. We choose this threshold because exact bounding box localization is not crucial for our application.

2) Edge Detection: Line detection describes the task of detecting the position of a line in the pixel space and it's type, which is used by previous approaches. In contrast, edge detection is the task of detecting edges between two objects or nodes. An edge detection metric should reflect the task as independently as possible from node detection. This is inherently difficult, because an edge is always associated with two nodes. If one of the nodes is missing, the edge will be missing as well. For a false positive node, there may be edges connected to that node that would never exist if the node had not been detected in the first place. Several commonly used metrics are described in Section II-B. However, the length of lines on a P&ID diagram does not necessarily correlate with the actual length of pipes or other components, and thus the metrics for road network extraction is not suitable to evaluate the quality of reconstructed graphs in this domain. In case of the engineering diagram digitization, where the connections are distinct and finite, mR@K is also not suited for the digitization problem. Thus, we propose a metric for edge detection mean Average Precision (edge mAP) using the Hungarian matching algorithm as depicted in Fig. 5. The Hungarian matching algorithm is used to solve assignment problems by optimally pairing elements from two sets while

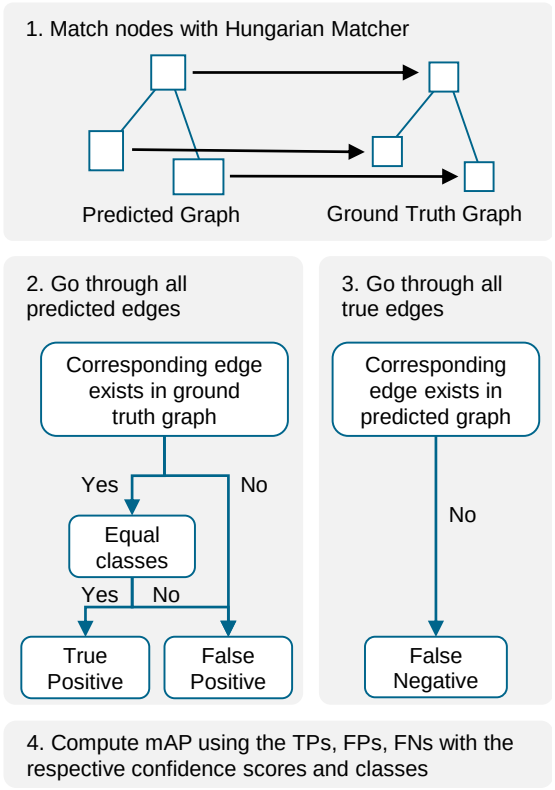


Fig. 5: Description of the implemented metric for calculating the edge mAP.

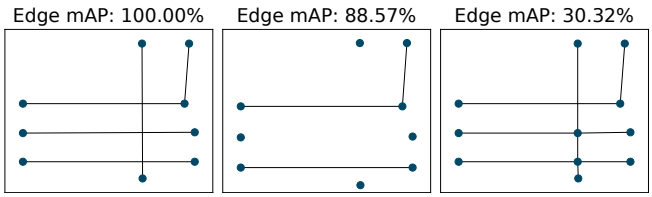


Fig. 6: Exemplary values of the edge mAP metric with the ground truth graph on the left and two other graphs in the middle and on the right. The graph in the middle misses two edges, and the one on the right adds several edges by falsely predicted crossing nodes.

minimizing the overall cost to match nodes from one graph to another using the nodes bounding boxes to use the gIoU as the cost function. Average precision (AP) is calculated using the precision-recall curve. A detailed description of the algorithm for edge mAP computation is Algorithm A1 in the appendix.

The evaluation metric is illustrated through an example calculation in Fig. 6. This example demonstrates that the metric places greater importance on correctly predicted edges and nodes than on not predicted or wrongly predicted ones, particularly when such predictions are made adjacent to incorrectly detected crossing nodes.

TABLE II: Amount of P&IDs and samples used for both training sets.

	Pre-Training Set	Training Set
Training Samples	170 944	44 019
Validation Samples	8998	2317
Real World P&IDs	0	60
Synthetic P&IDs	2000	500

F. Model Training

The Relationformer and Faster R-CNN are trained on a mix of the Synthetic 700 data and real world P&IDs, as presented in Table II. First, we pre-train both methods on a large set of 2000 synthetic P&IDs, which are patched and augmented multiple times to create a vast pool of 170 944 samples for training and 8998 samples for validation. Next, we fine-tune the models on a mix of the 60 real world P&IDs and a subset of 500 synthetic P&IDs. To increase the presence of real data in the training set, we augment each real world P&ID three times as much as each synthetic P&ID. This results in 44 019 samples for training and 2317 samples for validation in the second training phase. Approximately 37% of the training set comprised patches from real-world data, while 63% consisted of patches from synthetic plans. During this second phase of training, we stop when either the loss stabilizes or the validation loss begins to rise, indicating potential overfitting. Additional training details are listed in Section A2.

IV. Results

The results are presented in Table III. Our experiments on the patched OPEN100 data with the Relationformer model demonstrate good performance in detecting symbols (73.49%), nodes (82.18%) and connections (76.79%). When evaluated on full plans, the performance stays the same (around 1% increase or decrease). For the synthetic data, the symbol detection mAP has similar values after stitching the patches, while the stitching has a significant influence on node and edge detection, rising the node AP by 9.45% to 96.89% and lowering the edge mAP by 4.91% to 88.95%. In contrast, the modular digitization shows a more moderate performance, achieving decent symbol detection on the OPEN100 data on patches (86.58%) but struggling with node detection (52.15%) and edge detection (45.89%), similar to its performance on synthetic data.

Both methods demonstrate strong results on Dataset-P&ID, with modular digitization again showing better values only for symbol detection.

To investigate the impact of larger symbols, we conducted experiments on the OPEN100 data using an enlarged patch size to encompass bigger symbols within a single patch. Additionally, we performed experiments on a subset of the OPEN100 data that consisted only of plans where all symbols could be accommodated entirely within a patch. The results can be seen in Table IV. When increasing patch size from originally (1500, 1500) to (2000, 2000), which should facilitate better inclusion of larger symbols, the symbol and node detection

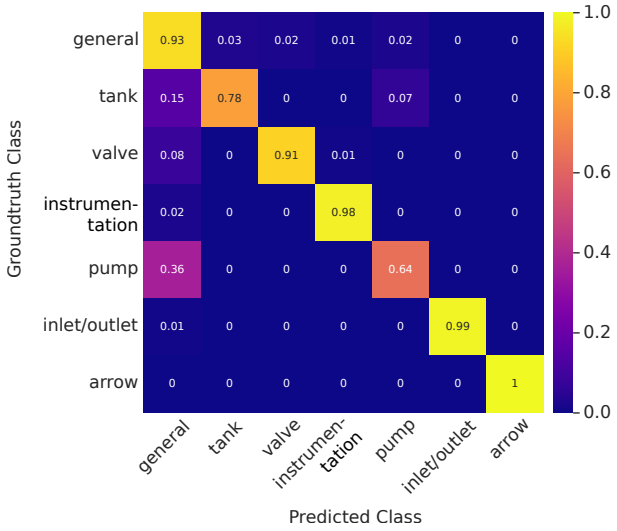


Fig. 7: Confusion matrix for the stiched symbol detection results of the Relationformer for the OPEN100 data.

value of the Relationformer drop by at least 10%, while the values of the Modular Digitization stay around the same. When using the small symbol subset, both methods show improved performance, with a gain of 9.49% for symbol detection by the Relationformer and 10.49% by the Modular Digitization. Moreover, the results show that the Modular Digitization's performance drops significantly from patched to stitched on the full OPEN100 data. However, its performance increases when using only the subset with small symbols. This suggests that the stitching process struggles with larger symbols.

The importance of real-world data in the training and test datasets is further underscored by an ablation study presented in Section A5, which demonstrates that the Relationformer's performance suffers significantly when trained on synthetic data only, particularly when evaluated on the OPEN100 dataset.

The performance of the Relationformer on the OPEN100 data is further investigated by examining the confusion matrix in Fig. 7. Notably, the 'general' class exhibits a high degree of confusion, with pumps being frequently misclassified as symbols belonging to this category. The confusion with the 'general' class is consistent with the other datasets (see Section A3 for the remaining confusion matrices).

To gain a better understanding of the detection capabilities and graph reconstruction quality, we have also visually inspected the predictions. Fig. 8 illustrates the merged graph for an entire plan. Overall, the graph is correctly constructed, though there are some noticeable misclassifications of instrumentation symbols and additional false positive diagonal connections between crossings and symbols.

V. Discussion

The Relationformer shows good results on every task, while the Modular Digitization shows especially bad results on the edge detection. The results highlight several challenges that

TABLE III: Performance comparison of the Modular Digitization and Relationformer on both test sets with higher values highlighted as bold. As in the Modular Digitzation the graph construction is done after patch merging, no values can be given for node AP and edge mAP for patches.

	Modular Digitization						Relationformer					
	Patched			Stitched			Patched			Stitched		
	Symbols mAP	Nodes AP	Edges mAP	Symbols mAP	Nodes AP	Edges mAP	Symbols mAP	Nodes AP	Edges mAP	Symbols mAP	Nodes AP	Edges mAP
PID2Graph OPEN100	**86.58**	-	-	**76.99**	52.14	45.89	73.49	82.18	76.79	73.14	**83.63**	**75.46**
PID2Graph Synthetic	**78.74**	-	-	74.15	85.16	50.26	78.62	87.44	93.86	**78.36**	**96.89**	**88.95**
Dataset-P&ID	**85.87**	-	-	**83.95**	89.32	85.46	80.32	96.59	92.46	76.69	**97.72**	**95.07**

TABLE IV: Ablation studies for the Relationformer and Modular Digitization for the OPEN100 data on different patch sizes and a test subset containing only objects that fit into a patch completely.

	Data	Patch Size	Symbols mAP	Nodes AP	Edges AP
Modular Digitization	Patched	(2000, 2000)	86.52	-	-
	Stitched	(2000, 2000)	76.74	57.49	55.08
	Patched, Small Symbol Subset	(1500, 1500)	86.51	-	-
	Stitched, Small Symbol Subset	(1500, 1500)	87.48	62.26	53.95
Relationformer	Patched	(2000, 2000)	62.23	50.11	69.88
	Stitched	(2000, 2000)	62.42	56.15	67.67
	Patched, Small Symbol Subset	(1500, 1500)	80.07	82.90	73.08
	Stitched, Small Symbol Subset	(1500, 1500)	82.74	83.80	75.16

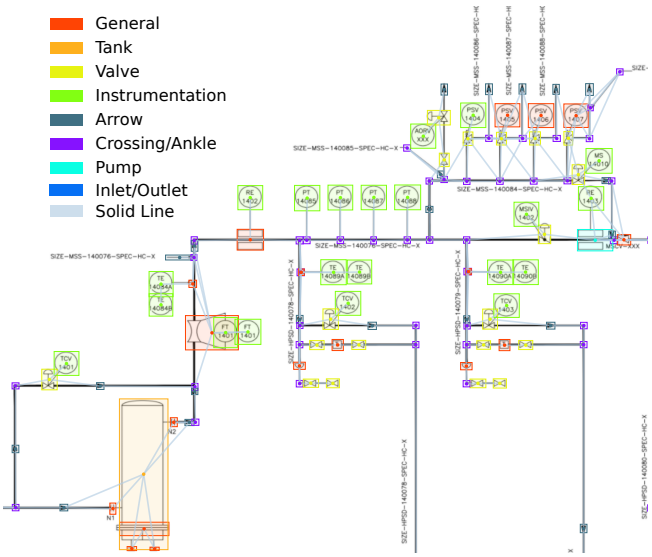


Fig. 8: Extract of a digitization result of the Relationformer for one merged P&ID from the OPEN100 test data. The legend shows the class names of the predictions.

need to be addressed in order to achieve accurate graph reconstruction and relation detection. The symbol detection module of the Modular Digitization however achieves better performance in correctly classifying symbols on the OPEN100 data.

The Modular Digitization has the disadvantage of being highly dependent on previous steps, where each step, especially line detection, is reliant on using good parameters. The Relationformer enables end-to-end training without significant adjustment after training. This property is expected to facilitate good generalization across different domains.

One of the primary difficulties lies in the patching and merging process. Errors occurred during the patching process itself, resulting in incorrect input data for our models used for training. When splitting the P&ID diagrams into patches, this can lead to the truncation of symbols. When only a fragment of a symbol remains, it can be difficult to distinguish it from other symbols or even lines, making it difficult for our models to train and evaluate on this data.

A problem with detecting symbols and differentiating between classes is the ”general” class. This class encompasses a diverse range of symbols, resulting in high intra-class variation. As a consequence, models struggle to generalize effectively across these disparate symbols, making evaluation and pinpointing specific issues within this category more challenging. The confusion matrices of the Relationformer support this assertion, as the general class is the class with the highest confusion across all datasets. The general detection and localization of symbols appear to be quite effective, as indicated by high node AP scores. However, improving the classification of symbols not seen during training and assigning more specific labels remains a focus for future work.

The metrics applied are based on classical symbol detection techniques and include a custom-defined metric. We used an IoU threshold of 0.5 to evaluate our methods, which is relatively low compared to other object detection tasks. A potential challenge in using a higher IoU threshold lies in the uncertainty of manually annotated data, as these annotations are less precise than the automatically generated bounding boxes for synthetic data. The utility of the edge metric warrants consideration, since it disproportionately penalizes incorrect edge predictions compared to missed edges. Furthermore, the edge detection metric relies on accurately matching nodes between the ground truth and predicted graphs based on bounding boxes. However, if a method accurately predicts edges, the metric will reflect this performance appropriately.

Additionally, our study underscores the importance of collecting and utilizing larger datasets with real-world diagrams including big symbols. Even though dashed or non-continuous lines are not present in the OPEN100 data, the results on the

synthetic data suggests that the Relationformer is also able to classify line types accurately.

VI. Conclusion

Leveraging the state-of-the-art Relationformer architecture, we propose a pipeline that simultaneously detects objects and their relationships from engineering diagrams. In this study, we address the challenge of digitizing engineering diagrams of complex technical systems, specifically Piping and Instrumentation Diagrams (P&IDs). By adapting the Relationformer architecture, we develop a robust pipeline that can simultaneously identify objects and their relationships within these diagrams. To facilitate evaluation and comparison, we introduce a novel, publicly available dataset and establish a set of meaningful metrics. Our approach yields impressive results on both real-world and synthetic P&IDs, outperforming a modular digitization approach. Notably, our method achieves an AP of 83.63% for node detection and an edge mAP of 75.46%, highlighting its effectiveness in accurately extracting valuable information from complex engineering diagrams.

Acknowledgment

We acknowledge the assistance of Llama 3.2 [38] throughout all sections of the paper to enhance the readability and clarity by improving formulations and overall English. The authors would like to thank Enrique Ríos Smits for his assistance in processing and labeling the data used in this work.

References

[1] M. Toghraei, *Piping and instrumentation diagram development*. Hoboken, NJ, USA: John Wiley & Sons Inc, 2019.

[2] N. Paganus, M. Luukkainen, K. Honkoila, and T. Karhela, "Automatic generation of dynamic simulation models based on standard engineering data," in *Proceedings of The 9th EUROSIM Congress on Modelling and Simulation, EUROSIM 2016*, ser. Linköping Electronic Conference Proceedings. Linköping University Electronic Press, 2018, pp. 776–782.

[3] J. M. Stürmer, M. Graumann, and T. Koch, "Demonstrating automated generation of simulation models from engineering diagrams," in *2023 International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2023, pp. 1156–1162.

[4] R. Rahul, S. Paliwal, M. Sharma, and L. Vig, "Automatic information extraction from piping and instrumentation diagrams," in *8th International Conference on Pattern Recognition Applications and Methods*, 2019.

[5] S. Mani, M. A. Haddad, D. Constantini, W. Douhard, Q. Li, and L. Poirier, "Automatic digitization of engineering diagrams using deep learning and graph search," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, 2020, pp. 673–679.

[6] M. Gada, "Object detection for p&id images using various deep learning techniques," in *2021 International Conference on Computer Communication and Informatics (ICCCI)*. IEEE, 2021, pp. 1–5.

[7] L. Jamieson, C. Francisco Moreno-García, and E. Elyan, "A review of deep learning methods for digitisation of complex documents and engineering diagrams," *Artificial Intelligence Review*, vol. 57, no. 6, 2024.

[8] J. K. Nurminen, K. Rainio, J.-P. Numminen, T. Syrjänen, N. Paganus, and K. Honkoila, "Object detection in design diagrams with machine learning," in *Progress in Computer Recognition Systems*, ser. Advances in Intelligent Systems and Computing, R. Burduk, M. Kurzynski, and M. Wozniak, Eds. Cham: Springer International Publishing, 2020, vol. 977, pp. 27–36.

[9] J.-M. Cha, K. Taewoo, T. Lee, and D. Mun, "Features recognition from piping and instrumentation diagrams in image format using a deep learning network," *Energies*, vol. 12, no. 23, p. 4425, 2019.

[10] C. F. Moreno-Garcia, P. Johnston, and B. Garkuwa, "Pixel-based layer segmentation of complex engineering drawings using convolutional neural networks," in *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2020, pp. 1–7.

[11] S. Paliwal, M. Sharma, and L. Vig, "Ossr-pid: One-shot symbol recognition in p&id sheets using path sampling and gcn," *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2021.

[12] B. C. Kim, H. Kim, Y. Moon, G. Lee, and D. Mun, "End-to-end digitization of image format piping and instrumentation diagrams at an industrially applicable level," *Journal of Computational Design and Engineering*, 2022.

[13] G. Kim and B. C. Kim, "Classification of functional types of lines in p&ids using a graph neural network," *IEEE Access*, vol. 11, pp. 73 680–73 687, 2023.

[14] M. F. Theisen, K. N. Flores, L. Schulze Balhorn, and A. M. Schweidtmann, "Digitization of chemical process flow diagrams using deep convolutional neural networks," *Digital Chemical Engineering*, vol. 6, p. 100072, 2023.

[15] H. Bhanbhro, Y. Kwang Hooi, W. Kusakunniran, and Z. H. Amur, "A symbol recognition system for single-line diagrams developed using a deep-learning approach," *Applied Sciences*, vol. 13, no. 15, p. 8816, 2023.

[16] L. Yang, J. Zhang, H. Li, L. Ren, C. Yang, J. Wang, and D. Shi, "A comprehensive end-to-end computer vision framework for restoration and recognition of low-quality engineering drawings," *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108524, 2024.

[17] J. Bayer, A. K. Roy, and A. Dengel, "Instance segmentation based graph extraction for handwritten circuit diagram images," *arXiv preprint arXiv:2301.03155*, 2023.

[18] S. Bickel, S. Goetz, and S. Wartzack, "Symbol detection in mechanical engineering sketches: Experimental study on principle sketches with synthetic data generation and deep learning," *Applied Sciences*, vol. 14, no. 14, p. 6106,

2024.

[19] L. Jamieson, C. F. Moreno-Garcia, and E. Elyan, "Towards fully automated processing and analysis of construction diagrams: Ai-powered symbol detection," *International Journal on Document Analysis and Recognition (IJDAR)*, 2024.

[20] S. He, F. Bastani, S. Jagwani, M. Alizadeh, H. Balakrishnan, S. Chawla, M. M. Elshrif, S. Madden, and M. A. Sadeghi, "Sat2graph: Road graph extraction through graph-tensor encoding," in *Computer Vision – ECCV 2020*, ser. Springer eBook Collection, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds. Cham: Springer International Publishing and Imprint Springer, 2020, vol. 12369, pp. 51–67.

[21] D. Belli and T. Kipf, "Image-conditioned graph generation for road network extraction," *arXiv preprint arXiv:1910.14388*, 2019.

[22] H. Li, G. Zhu, L. Zhang, Y. Jiang, Y. Dang, H. Hou, P. Shen, X. Zhao, S. A. A. Shah, and M. Bennamoun, "Scene graph generation: A comprehensive survey," *Neurocomput.*, vol. 566, no. C, Jan. 2024.

[23] R. Li, S. Zhang, and X. He, " SGTR+: End-to-End Scene Graph Generation With Transformer ," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, vol. 46, no. 04, pp. 2191–2205, Apr. 2024.

[24] S. Shit, R. Koner, B. Wittmann, J. Paetzold, I. Ezhov, H. Li, J. Pan, S. Sharifzadeh, G. Kaissis, V. Tresp, and B. Menze, "Relationformer: A unified framework for image-to-graph generation," in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds. Cham: Springer Nature Switzerland and Imprint Springer, 2022, vol. 13697, pp. 422–439.

[25] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," *arXiv preprint arXiv:2010.04159*, 2020.

[26] L. Xia, R. Liu, Y. Su, S. Mi, D. Yang, J. Chen, and Z. Shen, "Crop field extraction from high resolution remote sensing images based on semantic edges and spatial structure map," *Geocarto International*, vol. 39, no. 1, 2024.

[27] J. Biagioni and J. Eriksson, "Inferring road maps from global positioning system traces," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 2291, no. 1, pp. 61–71, 2012.

[28] A. V. Etten, D. Lindenbaum, and T. M. Bacastow, "Spacenet: A remote sensing dataset and challenge series," *arXiv preprint arXiv:1807.01232*, 2019.

[29] X. Li, L. Chen, J. Shao, S. Xiao, S. Zhang, and J. Xiao, "Rethinking the evaluation of unbiased scene graph generation," in *33rd British Machine Vision Conference 2022, BMVC 2022, London, UK, November 21-24, 2022*. BMVA Press, 2022.

[30] S. Paliwal, A. Jain, M. Sharma, and L. Vig, "Digitizepid: Automatic digitization of piping and instrumentation diagrams," in *Trends and Applications in Knowledge Discovery and Data Mining*, M. Gupta and G. Ramakrishnan, Eds. Cham: Springer International Publishing, 2021, pp. 168–180.

[31] E. Elyan, L. Jamieson, and A. Ali-Gombe, "Deep learning for symbols detection and classification in engineering drawings," *Neural networks : the official journal of the International Neural Network Society*, vol. 129, pp. 91–102, 2020.

[32] Y. Li, S. Xie, X. Chen, P. Dollar, K. He, and R. Girshick, "Benchmarking detection transfer learning with vision transformers," *arXiv preprint arXiv:2111.11429*, 2021.

[33] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, " Character Region Awareness for Text Detection ," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2019, pp. 9357–9366.

[34] JadedAI, "EasyOCR," 2023. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[35] S. B. Sandoval, "Baltakatei engineering drawing templates," https://gitlab.com/baltakatei/BK-2020-04, 2023, this work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by/4.0/.

[36] ISO Central Secretary, "Diagrams for the chemical and petrochemical industry — Part 2: Graphical symbols," International Organization for Standardization, Standard, Dec. 2012.

[37] E. I. C. Open100, "Open100 — nuclear energy," https://www.open-100.com/, 2024.

[38] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[39] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[40] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

APPENDIX

A1. Edge Metric Calculation

We calculate the metric for edge detection mean Average Precision (mAP) as described in Algorithm A1. The Hungarian matching algorithm [39] is used to match nodes (mapping M) from the ground truth graph G_{true} to the predicted graph G_{pred} using the nodes (denoted with V) bounding boxes. All edges $e_{u,v}$ from node u to node v of the predicted graph have a confidence score and a class. An edge $e_{\hat{u},\hat{v}}$ is an edge in the ground truth graph and also has a class. The Python library `scikit-learn` [40] is used to calculate the average precision.

Algorithm A1 Edge mAP Computation

Input: $G_{\text{true}}(V_{\text{true}}, E_{\text{true}})$, $G_{\text{pred}}(V_{\text{pred}}, E_{\text{pred}})$
Initialize:
(TP, FP, FN) $\leftarrow$ (empty list, empty list, empty list)
$M \leftarrow \text{HungarianMatcher}(V_{\text{true}}, V_{\text{pred}}) : V_{\text{true}} \rightarrow V_{\text{pred}}$
for each predicted edge $e_{u,v} \in E_{\text{pred}}$ **do**
  $e_{\hat{u},\hat{v}} \leftarrow e_{M^{-1}(u),M^{-1}(v)}$
  if $e_{\hat{u},\hat{v}} \in E_{\text{true}}$ **then**
    if class($e_{u,v}$) = class($e_{\hat{u},\hat{v}}$) **then**
      TP.insert((class($e_{u,v}$), conf_score($e_{u,v}$)))
    else
      FP.insert((class($e_{u,v}$), conf_score($e_{u,v}$)))
  else
    FP.insert((class($e_{u,v}$), conf_score($e_{u,v}$)))
for each true edge $e_{\hat{u},\hat{v}} \in E_{\text{true}}$ **do**
  $e_{u,v} \leftarrow e_{M(\hat{u}),M(\hat{v})}$
  if $e_{u,v} \notin E_{\text{pred}}$ **then**
    FN.insert(class($e_{\hat{u},\hat{v}}$))
$\text{mAP}_{\text{edge}} \leftarrow \frac{1}{|\text{classes}|} \sum_{c \in \text{classes}} \text{AP}_c(\text{TP, FP, FN})$
Return: mAP_{edge}

A2. Training

All training, testing, and experimental procedures were conducted on a single NVIDIA Quadro RTX 8000 graphics card with 48GB of memory. The relevant hyperparameters can be seen in Table A1 and Table A2. For both Relationformer and Faster R-CNN with the Modular Digitization, we employ a 0.95:0.05 training-validation split, allocating the majority of the data to training. This partitioning is deliberate, given the transformer network's propensity for requiring substantial amounts of data to achieve good performance.

A. Dataset Details

The datasets analyzed in this study prior to patching are summarized in Table A3, which reveals key characteristics. Specifically, the table reports the total number of symbols and edges, mean values for node and edge counts per plan, as well as the size distribution of symbols. Notably, real-world P&IDs exhibit a tendency towards increased complexity, characterized by higher mean numbers of nodes and edges per plan, as well as greater variance compared to synthetic data.

TABLE A1: Hyperparameters used to train the Relationformer.

Hyperparameter	Value
Batch Size	20
Image Size	(512, 512)
Epochs	80
Learning Rate	1e-4
Learning Rate Backbone	3e-5
Number of Queries	401
Backbone	ResNet-101
λ_{box}, λ_{cls}, λ_{card}, λ_{node}, λ_{edge}	2, 2, 1, 4, 3
Randomize Edge Directions	Yes

TABLE A2: Hyperparameters used to train Faster R-CNN for symbol detection.

Hyperparameter	Value
Batch Size	16
Image Size	(1024, 1024)
Epochs	40
Learning Rate	5e-4

A3. Further Analysis of General Class

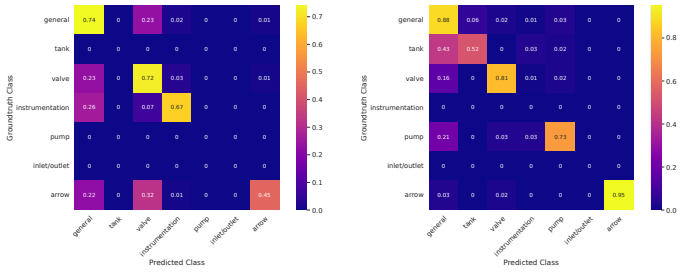
The confusion matrix of the Relationformer on the PID2Graph OPEN100 dataset is shown in Fig. 7, while the confusion matrices of the Relationformer on the PID2Graph Synthetic and Dataset-P&ID are shown in Fig. A1.

The confusion is the highest with the 'general' class for PID2Graph and Dataset-P&ID, with confusion ranging up to 43%.

To gain deeper insight into the diversity within each class, we extracted feature vectors from every symbol in our dataset using ResNet-101 with an input size of (224, 224). We then applied t-SNE to visualize these features in a 2D space (see Fig. A2). This analysis reveals that the 'general' class exhibits the highest degree of centralization and overlap with other classes across all datasets. This also suggests substantial intra-class variability for the 'general' category.

A4. Dataset-P&ID Comparison

The line detection precision of DigitizePID is not directly comparable to edge mAP, however we display both of them.

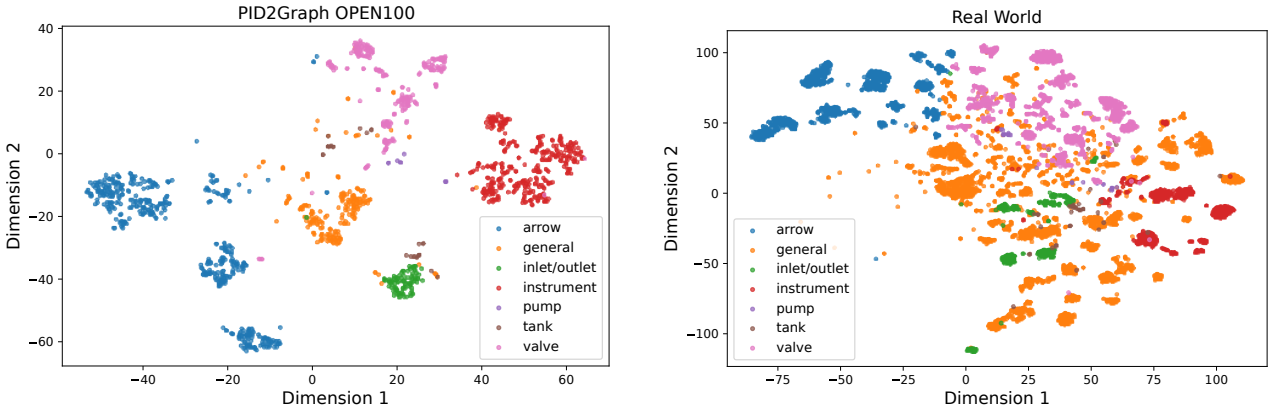


(a) (b)

Fig. A1: Confusion matrices for stitched symbol detection results of the Relationformer for (a) Dataset-P&ID and (b) PID2Graph Synthetic.

TABLE A3: Dataset statistics prior to splitting and augmentation. Real-world P&IDs exhibit significantly higher data variability compared to synthetic counterparts.

	# of Nodes per Plan	# of Edges per Plan	# of Symbols, Crossings & Ankles	# of Edges	Symbol Area (px) Mean ± Std	Usage
Synthetic 700	131 ± 14	133 ± 15	183 901	188 550	6071 ± 22 059	Train
Real World	497 ± 338	457 ± 338	17 218	14 649	5776 ± 54 328	Train
Dataset-P&ID	452 ± 86	505 ± 91	51 182	137 449	3350 ± 4291	Test
PID2Graph Synthetic	131 ± 14	133 ± 15	23 323	23 994	4208 ± 8993	Test
PID2Graph OPEN100	382 ± 125	381 ± 126	2173	2100	13 644 ± 69 234	Test
PID2Graph OPEN100 Subset	414 ± 44	412 ± 47	793	765	12 442 ± 60 297	Test



(a) (b)

Fig. A2: 2D t-SNE visualization of features by class for (a) PID2Graph OPEN100 and (b) Real World.

TABLE A4: Performance comparison of the Modular Digitization, the Relationformer and DigitizePID on the synthetic Dataset-P&ID dataset.

	Dataset-P&ID [30], Stitched			
	Symbols mAP	Nodes AP	Edges mAP	Lines Accuracy
Modular Digitization	69.82	85.42	85.46	-
Relationformer	43.04	75.85	95.07	-
DigitizePID [30]	92.50	-	-	91.13

Our evaluation methods required converting the Dataset-P&ID [30] data into a graph structure (as described in Section 3.4). Notably, we used an Intersection-over-Union (IoU) threshold of 75% for calculating mAP values, which is higher than the 50% threshold used in our other experiments. The line detection precision of DigitizePID is not directly comparable to our edge mAP results, but both metrics are presented for reference.

Despite this, the results show that both Modular Digitization and Relationformer perform poorly compared to DigitizePID, likely due to differences in IoU thresholds and bounding box styles. Specifically, the loose-fitting bounding boxes in Dataset-P&ID, combined with smaller symbol sizes (cf. Section A2-A), resulted in non-detections at this higher IoU. However, our results indicate that the Relationformer's edge detection is robust and effective.

TABLE A5: Relationformer performance comparison when trained on synthetic data only versus the mix of synthetic and real-world data.

Trained on			Patched		
Synthetic	Mix	Test Dataset	Symbols mAP	Nodes AP	Edges AP
✓		PID2GRAPH OPEN100	26.67	34.35	44.49
✓		PID2GRAPH Synthetic	63.20	73.71	88.02
✓		Dataset-P&ID	36.36	72.45	84.88
✓	✓	PID2GRAPH OPEN100	73.49	82.18	76.79
✓	✓	PID2GRAPH Synthetic	78.62	87.44	93.86
✓	✓	Dataset-P&ID	80.32	96.59	92.46

A5. Ablation

To underscore the value of training on real-world data, we conducted a comparative evaluation of the Relationformer model, examining its performance when trained exclusively on synthetic P&ID data versus when trained on a combination of synthetic data and the mix of synthetic and real-world data. The results in Table A5 show a consistent improvement in performance when the model is trained on the mixed dataset. Notably, the largest performance gap is observed when comparing the results on the OPEN100 dataset, highlighting the benefits of incorporating diverse and realistic training data.